
A03**Collections****Algorithmic Thinking**
ALD (MTD.ba VZ)
SS 2026

Submission via Moodle**24 Points (+4 Bonus)**

This assignment applies the data structures from Session 5 in two game scenarios. The key design idea in both exercises: **build a specialised data structure** that enforces the game's rules internally, so the game logic stays simple.

Submission Requirements:

- Read the [Organizational Guidelines](#) on Moodle, and adhere to them!
- Submit a PDF document containing your solution idea, code, and explanations for each exercise
- Include all source files in a ZIP archive (reuse your UE6 Maven project)
- Use the naming convention: `ALD_A03_LASTNAME.pdf` and `ALD_A03_LASTNAME.zip`

JavaFX Bonus (+4 points): A console implementation earns full marks (24 pts). If you implement a visual JavaFX version of either game, you earn up to 4 additional bonus points (partial credit: 2 pts for a static visual, 4 pts for a working interactive/animated version).

Exercise 1 (12 points) Towers of Hanoi — Stack with a Built-In Rule

The Towers of Hanoi puzzle has three pegs (A, B, C) and n discs of different sizes, all starting on peg A (largest at bottom). Move all discs to peg C following two rules: (1) move only one disc at a time; (2) never place a larger disc on top of a smaller one.

Each peg is naturally a **Stack**. The key insight: instead of checking the rule manually in your game logic, encode it *into the stack itself*.

Part A — PegStack (5 points)

Create `PegStack.java`. It must **extend** `Stack` and **override** `push()` to add the disc constraint — all other methods are inherited unchanged:

```
1  public class PegStack extends Stack {
2
3      public PegStack(int capacity) { super(capacity); }
4
5      // Throws RuntimeException if disc is larger than the current top
   disc.
6      // Pushing onto an empty peg always succeeds.
7      @Override
8      public void push(int disc) { ... }
9      // pop(), peek(), isEmpty(), size(), toString() are inherited from
   Stack
10 }
11
```

In your PDF: show `PegStack.java` and demonstrate that `push(3)` onto a peg that already holds disc 1 throws an exception, while `push(1)` onto an empty peg succeeds.

Part B — HanoiSolver and HanoiGame (5 points)

HanoiSolver.java (auto-solver, 2 pts): Use `PegStack` for all three pegs. The recursive solver should call `pop()` and `push()` directly — *no manual disc-size check* is needed, because `PegStack` already enforces it.

Run `HanoiSolver` for $n = 1, 2, 3, 4, 5$ and include a table in your PDF showing `getMoveCount()` vs. $2^n - 1$. All five must match.

HanoiGame.java (interactive mode, 3 pts): Let the player move discs manually. Your game should:

- Prompt for source and destination peg (e.g. A C)
- Pop the disc from the source peg, then try to push it onto the destination peg
- If `PegStack.push()` throws, *put the disc back* on the source peg and print the error message
- Print the state of all three pegs after each attempt (valid or not)

- Detect when all discs are on peg C and print a congratulations message with the move count

Include a console transcript showing at least one rejected invalid move and the final solved state.

Part C — Written Analysis (2 points)

Answer in 3–5 sentences: “*Why is a Stack the natural data structure for a Hanoi peg? What happens if you replace each PegStack with a Queue?*”

Exercise 2 (12 points) Snake — A List that Moves Itself

Implement a turn-based Snake game. As in Exercise 1, the key design goal is to encode the game's core behaviour *into the data structure*: the list should know how to **move** (shift the snake one step) and how to **feed** (grow the snake when it eats food).

Part A — Extend MyLinkedList (3 points)

Your MyLinkedList from UE6 is missing tail removal. Add:

```
1 public int removeLast() { ... } // throw RuntimeException if empty
2
```

In your PDF: explain **why** `removeLast()` is **O(n)** and what data structure change would make it **O(1)**.

Part B — SnakeList (4 points)

Create SnakeList.java. It must **extend** MyLinkedList and add three methods — `contains()`, `size()`, and `toString()` are all inherited. Positions are encoded integers (`row * gridSize + col`) — the grid details belong in SnakeGame, not here.

```
1 public class SnakeList extends MyLinkedList {
2     // First position = head, last position = tail
3     public SnakeList(int... positions) { ... }
4
5     // Shifts snake one step: adds newHeadPos at front, removes tail.
6     // Returns the position the tail vacated.
7     public int move(int newHeadPos) { ... }
8
9     // Feeds snake: adds newHeadPos at front, tail stays (snake grows)
10    .
11
12    public void feed(int newHeadPos) { ... }
13
14    public int getHead() { ... } // current head position
15    // contains(), size(), toString() inherited from MyLinkedList
16 }
```

In your PDF: demonstrate `move()` and `feed()` on a 3-segment snake.

Part C — SnakeGame (5 points)

Create SnakeGame.java on a 10×10 grid. Encode each cell as `row * 10 + col`. Use SnakeList as the snake — your game should **only call** `snake.move()` **or** `snake.feed()`, never `raw.addFirst/removeLast`.

- **Initialization:** 3-segment snake in the centre; food at a random empty cell
- **Turn loop:** read direction (W/A/S/D), then:
 1. Compute the new head position from the current head and direction
 2. Wall collision (out of bounds) $\rightarrow$ game over
 3. Self-collision (`snake.contains(newHead)`) $\rightarrow$ game over
 4. Food: call `snake.feed(newHead)`, place new food
 5. Otherwise: call `snake.move(newHead)`
- **Board display:** @ = head, 0 = body, * = food, . = empty
- **Win:** snake fills all 100 cells

Include a console transcript: initial board, ≥ 3 turns including one food pickup, and a game-over or win screen.
